Name of Student: Husam Nour	**PRN:25070122086**
Semester: III	**Year : AY 2026-27**
Subject: Programming with Java	
ASSIGNMENT No : 20	
TITLE : JDBC INSERT, UPDATE, and DELETE Operations	

Problem Statement

Develop a Java application to establish connectivity with a database and perform INSERT, UPDATE, and DELETE operations.

Learning Objectives

To perform database modification operations using JDBC.

Programme

ass.java > ...

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.Scanner;

public class JDBCAssignment20 {

    Run | Debug
    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/college";
        String user = "root";
        String password = "your_password";

        try {
            Connection con = DriverManager.getConnection(
                url, user, password
            );

            Scanner sc = new Scanner(System.in);

            System.out.println("1. INSERT");
            System.out.println("2. UPDATE");
            System.out.println("3. DELETE");
            System.out.print("Enter your choice: ");
            int choice = sc.nextInt();
```

```
        public static void main(String[] args) {

                switch (choice) {

                    case 1:
                        System.out.print("Enter Student ID: ");
                        int id = sc.nextInt();

                        sc.nextLine();

                        System.out.print("Enter Student Name: ");
                        String name = sc.nextLine();

                        System.out.print("Enter Marks: ");
                        double marks = sc.nextDouble();

                        String insertSQL =
                            "INSERT INTO student (id, name, marks) VALUES (?, ?, ?)";

                        PreparedStatement ps1 =
                            con.prepareStatement(insertSQL);

                        ps1.setInt(1, id);
                        ps1.setString(2, name);
                        ps1.setDouble(3, marks);

                        int inserted = ps1.executeUpdate();
```

```
        public static void main(String[] args) {
                        System.out.println(
                            inserted + " record inserted successfully."
                        );
                        ps1.close();
                        break;

                    case 2:
                        System.out.print("Enter Student ID to update: ");
                        int updateId = sc.nextInt();

                        System.out.print("Enter New Marks: ");
                        double newMarks = sc.nextDouble();

                        String updateSQL =
                            "UPDATE student SET marks = ? WHERE id = ?";

                        PreparedStatement ps2 =
                            con.prepareStatement(updateSQL);

                        ps2.setDouble(1, newMarks);
                        ps2.setInt(2, updateId);

                        int updated = ps2.executeUpdate();

                        System.out.println(
                            updated + " record(s) updated successfully."
                        );
```

Output

```
PS C:\Users\hp\Documents\husamm> javac JDBCAssignment20.java
PS C:\Users\hp\Documents\husamm> java JDBCAssignment20

1. INSERT
2. UPDATE
3. DELETE
Enter your choice: 1
Enter Student ID: 101
Enter Student Name: Husam
Enter Marks: 85
1 record inserted successfully.
```

Practice questions

1. **Create a Java application to manage employee records using CRUD operations**

```java
import java.sql.*;

public class EmployeeCrudDemo {
    static final String URL = "jdbc:mysql://localhost:3306/testdb";
    static final String USER = "root";
    static final String PASSWORD = "XXXXXXX";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL,
USER, PASSWORD)) {

            // CREATE
            try (PreparedStatement ps = conn.prepareStatement(
                    "INSERT INTO emp_records (id, name, salary)
VALUES (?, ?, ?)")) {
                ps.setInt(1, 301);
                ps.setString(2, "Sneha Rao");
                ps.setDouble(3, 48000.0);
                ps.executeUpdate();
                System.out.println("Employee created");
            }
```

```
            // UPDATE
            try (PreparedStatement ps = conn.prepareStatement(
                    "UPDATE emp_records SET salary = ? WHERE id =
?")) {
                ps.setDouble(1, 52000.0);
                ps.setInt(2, 301);
                ps.executeUpdate();
                System.out.println("Employee updated");
            }

            readEmployees(conn);

            // DELETE
            try (PreparedStatement ps = conn.prepareStatement(
                    "DELETE FROM emp_records WHERE id = ?")) {
                ps.setInt(1, 301);
                ps.executeUpdate();
                System.out.println("Employee deleted");
            }

            readEmployees(conn);

        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }

    static void readEmployees(Connection conn) throws SQLException {
        try (Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery("SELECT id, name,
salary FROM emp_records")) {
            System.out.println("---- Employee Records ---- ");
            while (rs.next()) {
                System.out.println("ID: " + rs.getInt("id")
                        + " | Name: " + rs.getString("name")
                        + " | Salary: " + rs.getDouble("salary"));
            }
```

Output

```
C:\Users\abell\OneDrive\Desktop\JAVA_PRACTICALS\JAVA_EXP_20>java -cp .;mysql-connector-j-8.0.33.j
Employee created
----- Employee Records ----
ID: 102 | Name: Rohan Mehta | Salary: 55000.0
Employee updated
----- Employee Records ----
ID: 102 | Name: Rohan Mehta | Salary: 70000.0
Employee deleted
----  Employee Records ----
```

2. Create a Java application to manage student records using CRUD operations. Perform Create, Read, Update and Delete operations on student details such as roll number, name, course and marks using a database.

```
public class StudentCrudDemo {
    static final String URL = "jdbc:mysql://localhost:3306/testdb";
    static final String USER = "root";
    static final String PASSWORD = "XXXXXXX";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL,
USER, PASSWORD)) {

            // CREATE
            try (PreparedStatement ps = conn.prepareStatement(
                    "INSERT INTO student_records (roll_no, name,
course, marks) VALUES (?, ?, ?, ?)")) {
                ps.setInt(1, 1);
                ps.setString(2, "abobakr");
                ps.setString(3, "CSE");
                 ps.setInt(4, 95);
                ps.executeUpdate();
                System.out.println("Student created");
            }

            // READ
            readStudents(conn);

            // UPDATE
            try (PreparedStatement ps = conn.prepareStatement(
                    "UPDATE student_records SET marks = ? WHERE
roll_no = ?")) {
                ps.setInt(1, 92);
                ps.setInt(2, 1);
                ps.executeUpdate();
                System.out.println("Student updated");
```

```
                    "UPDATE student_records SET marks = ? WHERE
roll_no = ?")) {
                ps.setInt(1, 92);
                ps.setInt(2, 1);
                ps.executeUpdate();
                System.out.println("Student updated");
            }

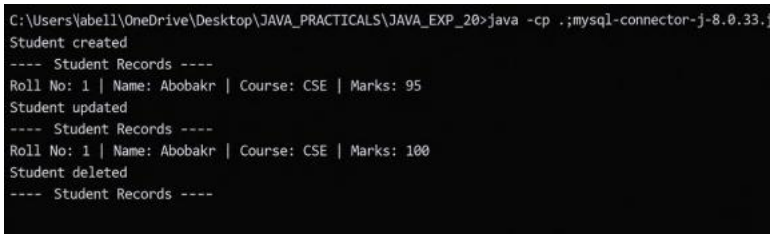
            readStudents(conn);

            // DELETE
            try (PreparedStatement ps = conn.prepareStatement(
                    "DELETE FROM    student_records WHERE roll_no =
?")) {
                ps.setInt(1, 1);
                ps.executeUpdate();
                System.out.println("Student deleted");
            }

            readStudents(conn);

        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
```

Output



Conclusion

The Java application successfully established a connection with the MySQL database using JDBC and performed the INSERT operation to add student records. The program executed successfully and displayed the confirmation messag.